

10 — Python imports, type annotations, and parsing

The question

What do `S1` imports, from `__future__` import `annotations`, and `parse` mean in the `WO Table 4` script?

This guide uses `Code_Python_rogp39/Case2-WO-Table4-Reproduction/wo_eq36_fullscan_IK_casadi.py`. The small examples explain Python behavior; they do not run the process model. The version-specific behavior below was checked with Python 3.9.25.

1. Type annotations describe expected data

```
def square(x: float) -> float:
    return x * x
```

`x: float` describes the expected input type. `-> float` describes the expected return type. The calculation is still `x * x`. Python does not automatically enforce these hints or convert the data: `square(3)` returns the integer 9. See [Python type hints](#)^[1].

In the `WO` script, read this parameter one piece at a time:

```
p: dict[str, float] | None = None
```

Part	Meaning
<code>p</code>	Parameter name
<code>dict[str, float]</code>	Dictionary with string keys and float values, such as <code>{"rho": 50.0}</code>
<code> None</code>	The type hint also allows <code>None</code>
<code>= None</code>	If the caller omits <code>p</code> , its default value is <code>None</code>

A dictionary can contain zero, one, or many key–value pairs. In `dict[str, float]`, `str` describes the keys and `float` describes the values; the two type names do not mean there is only one pair. For example:

```
p = {"rho": 50.0, "k": 0.2, "volume": 60.0}
print(len(p))      # 3
print(p["rho"])     # 50.0
print(p["k"])       # 0.2
```

These are illustrative entries, not a complete validated `WO` parameter set. Each key is a string and each value here is a float. The annotation does not prescribe particular key names, their count, or physical units; the function body may impose additional requirements. `{}` is an empty dictionary, while `None` means no dictionary was supplied. They are different values.

Likewise, `sc: list[dict[str, Any]]` describes a list of scenario dictionaries. `Any` leaves value types unrestricted; a record can contain float flows and integer indices. `-> np.ndarray` describes an array return value.

2. Why the future import is needed here

2.1 A number and a piece of text are different

Start with these two lines. The only difference is the quotation marks:

```
number = 1 + 2
text = "1 + 2"

print(number)      # 3
print(type(number)) # <class 'int'>
print(text)        # 1 + 2
print(type(text))  # <class 'str'>
```

For `number`, Python performs the addition and stores the result 3. For `text`, Python stores the characters `1 + 2` as a string. **Saving those characters does not perform the addition, now or automatically later.**

A separate instruction can ask Python to execute the saved text as an expression. Here `eval()` does that for our fixed arithmetic example:

```
text = "1 + 2"
result = eval(text)
print(result) # 3
print(text)   # 1 + 2
```

`result` receives 3; `text` itself remains the string `"1 + 2"`. This example explains the difference between storing text and asking Python to execute it. You do not need to add `eval()` to the WO script.

2.2 Apply that distinction to the parameter annotation

In `p: dict[str, float] | None = None`, the part after the colon and before `= None` describes the expected type of `p`: **a dictionary with string keys and float values, or None**. The `|` means “or” here. The final `= None` separately specifies the default value if no argument is supplied.

Without the future import, Python 3.9 tries to turn this annotation into a type description it can work with when defining the function. That requires support for `|` between these types. Python 3.9 does not have that support, so the definition raises `TypeError`. This use of `|` was introduced in [Python 3.10](#)^[2].

With `from __future__ import annotations`, Python 3.9 instead keeps this annotation as the string `"dict[str, float] | None"`. Just as saving `"1 + 2"` does not require addition, saving this text does not require performing the unsupported `|` operation. The function can therefore be defined and called. See [PEP 563](#)^[3].

Run the following as a complete, separate Python 3.9 script:

```

from __future__ import annotations

def identity(p: dict[str, float] | None = None):
    return p

parameters = {"rho": 50.0, "k": 0.2}
result = identity(parameters)
print(result)                # {'rho': 50.0, 'k': 0.2}
print(type(result).__name__)  # dict

annotation = identity.__annotations__["p"]
print(annotation)            # dict[str, float] | None
print(type(annotation).__name__) # str

```

There are two separate things to inspect: `result` is the real dictionary returned by the function; `annotation` is the saved text describing the parameter. `identity.__annotations__["p"]` simply retrieves that description. **The dictionary has not become a string.**

2.3 Is it equivalent to a comment?

It resembles a comment in that it does not immediately perform the `|` operation, but it is not discarded. A normal `#` comment is not stored as the function's annotation; this description is stored in `__annotations__` and can be retrieved by tools. The future import also does not add automatic checking of the argument values.

2.4 When can an error still occur? (Optional)

An ordinary `identity(parameters)` call uses the dictionary and runs `return p`; it does not ask Python to convert the saved annotation text into type information. But `typing.get_type_hints(identity)` makes that additional request. On Python 3.9, it then encounters the unsupported `|` operation and raises `TypeError`. This is a separate action, like explicitly calling `eval(text)` in the arithmetic example. **A stored string does not execute itself, and the future import does not upgrade Python.**

If a Python 3.9 program needs that extra tool, the compatible spelling below expresses the same intended type. This is an optional alternative; the original WO script is not being changed.

```

from typing import Optional

def identity(p: Optional[dict[str, float]] = None):
    return p

```

`Optional[dict[str, float]]` also means “this dictionary type or None.” Neither spelling restricts the dictionary to a single key–value pair.

`__future__` is a special standard-library module. Python recognizes `from __future__ import annotations` as an instruction to enable this annotation behavior in the file. *Dunder* means *double underscore*: `__future__` has two underscores at each end. Put the statement before ordinary imports; a module docstring and comments may precede it. See [future statements](#) ^[4].

3. Read the S1 imports by their job

S1 makes tools available. These import lines do not start the WO optimization.

Import	Use in this script
<code>import argparse</code>	Read options such as <code>--output-dir</code>
<code>import csv</code>	Read historical MATLAB results and write result/comparison tables
<code>from datetime import datetime, timezone</code>	Record UTC dates and create timestamped run names
<code>import hashlib</code>	Record SHA-256 file fingerprints for source tracking
<code>import json</code>	Save configuration, versions, and run records
<code>from pathlib import Path</code>	Locate files, build paths, and create output directories
<code>import platform</code>	Record the Python version
<code>import time</code>	Measure solver elapsed time with <code>perf_counter()</code>
<code>from typing import Any</code>	Supply <code>Any</code> for type annotations
<code>import casadi as ca</code>	Build symbolic variables, objectives, and constraints; call IPOPT
<code>import numpy as np</code>	Work with numerical arrays, initial guesses, and results

`import csv` gives names such as `csv.DictWriter`. `from pathlib import Path` allows `Path(...)` directly. `as np` gives NumPy a short name. These are three common import forms.

```
V = ca.MX.sym("V") # Symbolic decision variable
V0 = np.array([60.0]) # Numerical initial value
```

These two lines require the CasADi and NumPy imports above. CasADi represents the optimization model; NumPy handles numerical data. Later, `ca.nlpsol(..., "ipopt", ...)` selects IPOPT as the solver.

4. Parse means read according to rules

When parsing Python text such as `a = 1 + 2`, a parser identifies an assignment and an addition. Executing it calculates 3 and assigns it to `a`.

In this script, `parser.parse_args()` instead reads command-line options and stores their values in `args`. The options `--output-dir` and `--historical-csv` become `args.output_dir` and `args.historical_csv`; omitted options are `None`. Here `type=Path` actually converts a supplied value to a path object; it is not a type annotation. See [argparse](#) ^[5].

Run the [small standalone example](#) from the handbook root. It requires Python 3.9+ and no external packages; it creates no output directory or simulation results.

```
python examples/10_python_annotations_and_parsing.py --output-dir results/demo
```

Expected output:

```
p value: {'rho': 50.0}
p type: dict
annotation: dict[str, float] | None
annotation type: str
output path: results/demo
```

The dictionary is unchanged, its type annotation is stored as text, and the command-line path has been read. The real WO script then passes that path to `run_sweep()` to begin its calculations.

5. Read the special names in the example

5.1 What does `identity.__annotations__["p"]` retrieve?

Read this expression from left to right. `identity` is the function object; the dot accesses its `__annotations__` attribute, a dictionary of type annotations; `["p"]` selects the entry whose key is the parameter name `p`. It does not call `identity` or retrieve a supplied argument value. Python documents these attributes in its [data model](#) ^[6].

With the `identity` definition above and the future import, Python 3.9 produces:

```
print(identity.__annotations__)
# {'p': 'dict[str, float] | None'}

annotation = identity.__annotations__["p"]
print(annotation)          # dict[str, float] | None
print(type(annotation))    # <class 'str'>

value = identity({'rho': 50.0})
print(value)               # {'rho': 50.0}
print(type(value))        # <class 'dict'>
```

`annotation` holds the saved description; `value` holds the actual dictionary. The string key `"p"` selects the description even when no function call has occurred. The final `= None` belongs to the parameter default, so it is absent from the annotation string.

5.2 What does `__name__` mean here?

Look at the object before the dot. `identity.__name__` is the function's name. `type(annotation)` returns the type object `str`; accessing that object's `__name__` returns its short name, the string `"str"`.

```
print(identity.__name__)      # identity
print(type(annotation))      # <class 'str'>
print(type(annotation).__name__) # str
```

In the script's entry-point condition, bare `__name__` instead refers to the current module's name. A function's name, a type's name, and a module's name describe different objects.

5.3 Why two underscores at both ends?

`__name__` and `__annotations__` use two underscores on each side, with no spaces. This is the *dunder* naming pattern. Python reserves this pattern for documented special names; the [identifier rules](#)^[7] explain the convention.

Name	Established purpose
<code>__name__</code>	Name of the relevant module, function, or class
<code>__annotations__</code>	Stored type annotations
<code>__init__</code>	Method used to initialize a newly created instance

The underscores are part of the exact name, not an operation that adds a capability. `identity.name` does not automatically mean `identity.__name__`, and inventing `__my_setting__` gives it no automatic behavior. Use ordinary names such as `parameters` or `main` for your own variables and functions. A method beginning with two underscores but not ending with two, such as `__helper`, follows a different class naming rule.

5.4 Why write `if __name__ == "__main__":`?

The following independent teaching file, `demo.py`, makes the distinction visible. See Python's [top-level script environment](#)^[8].

```
# Save this separate teaching example as demo.py.
print("module name:", __name__)

def main():
    print("Starting demo")

if __name__ == "__main__":
    main()
```

Run these commands separately from the directory containing `demo.py`:

Command	Printed output
<code>python demo.py</code>	module name: <code>__main__</code> Starting demo
<code>python -c "import demo"</code>	module name: <code>demo</code>

Direct execution gives the module the name `"__main__"`, so the condition is true and calls `main()`. Importing it as `demo` makes the condition false. Import still executes the unguarded top-level `print` and defines the function; it skips only the guarded call. The chapter's standalone script has no such unguarded print, so importing it produces no output.

`main` is an ordinary function name chosen by the author; Python does not call it just because of that name. `def main():` defines it, while `main()` calls it. `"__main__"` is a separate string identifying the entry module, and `==` compares values. You could rename the function to `run_demo` if you also change its call.

These annotation, name, and direct-run/import examples were checked with Python 3.9.25. The complete chapter script follows.

6. Complete runnable example

Below is the complete source of `examples/10_python_annotations_and_parsing.py`. Save it as that file and run the Section 4 command with Python 3.9+. It uses only the standard library and prints values without creating files or running optimization.

```

"""Chapter 10 teaching example. Python 3.9+; standard library only.

Run: python examples/10_python_annotations_and_parsing.py --output-dir results/demo
Reads an option and prints values; does not create files or run optimization.
"""
from __future__ import annotations

import argparse
from pathlib import Path

def identity(p: dict[str, float] | None = None):
    return p

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--output-dir", type=Path)
    args = parser.parse_args()

    p = {"rho": 50.0}
    value = identity(p)
    annotation = identity.__annotations__["p"]
    print("p value:", value)
    print("p type:", type(value).__name__)
    print("annotation:", annotation)
    print("annotation type:", type(annotation).__name__)
    print("output path:", args.output_dir.as_posix() if args.output_dir is not None else None)

if __name__ == "__main__":
    main()

```

References

1. <https://docs.python.org/3.9/library/typing.html>
2. <https://docs.python.org/3.10/whatsnew/3.10.html#pep-604-new-type-union-operator>
3. <https://peps.python.org/pep-0563/>
4. https://docs.python.org/3.9/reference/simple_stmts.html#future-statements
5. <https://docs.python.org/3.9/library/argparse.html>
6. <https://docs.python.org/3.9/reference/datamodel.html#user-defined-functions>
7. https://docs.python.org/3.9/reference/lexical_analysis.html#reserved-classes-of-identifiers
8. https://docs.python.org/3.9/library/__main__.html